

**A PROJECT REPORT
ON
CLOUD-FORENSICS AUTOMATION FOR
RETAIL BREACHES**

**Under the guidance of
Ms. Munisah Muneer
Assistant Professor
Department of Computer Science**

Submitted by

Joel Abin

23BCCE108

**BACHELOR OF COMPUTER APPLICATIONS
Cloud Computing, Cybersecurity & Ethical Hacking**



YENEPOYA
Deemed to be University

**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE &
MANAGEMENT**

(A constituent unit of Yenepoya Deemed to be University)

Deralakatte, Mangaluru – 575018, Karnataka, INDIA

CERTIFICATE

This is to certify that the project work entitled “Cloud-Forensics Automation for RetailBreaches” has been successfully carried out at “**IBM Technology Company**” by **Joel Abin** (Reg. No. 23BCCE108), student of **3rd Year BCA Cybersecuity, Ethical Hacking & Cloud Computing** under the supervision and guidance of **Ms. Munisah Muneer**, Assistant Professor ,Department of Computer Science, **Yenepoya Institute of Arts, Science, Commerce and Management**. This dissertation is submitted in partial fulfilment for the award of degree in **Bachelor of Computer Applications by Yenepoya (Deemed to be university)** during academic year 2025-26.

Internal Guide:

Chairperson:

Internal Examiner:

External Examiner:

PRINCIPAL

Dr. Jeevan Raj

**The Yenepoya Institute of Arts,
Science, Commerce and Management
(Deemed To be university)**

Submitted for the Viva-Voce

Place: Mangalore

Examination held on:

DECLARATION

I, **Joel Abin**, a student of **BCA Cybersecurity, Ethical Hacking & Cloud Computing** at **Yenepoya Institute of Arts, Science, Commerce, and Management, Balmatta, Mangalore**, affiliated with **Yenepoya (Deemed to be University)**, hereby declare that this internship report titled **Cloud-Forensics Automation for Retail Breaches** is a genuine and original record of the work undertaken by me as part of my academic curriculum.

This report documents the knowledge, skills, and practical experience acquired during my internship at IBM Technologies. It includes methodologies, analytical processes, and investigative approaches aligned with recognized industry standards in digital forensics, incident response, and cybersecurity investigations.

I extend my sincere gratitude to **Sumith Kumar Shukla, IBM Program Head**, for his valuable guidance, mentorship, and support throughout the internship period. I also express my appreciation to IBM for providing an enriching environment and exposure to real-world cybersecurity operations.

Furthermore, I acknowledge the support and encouragement received from my institutional guide **Ms. Munisah Muneer**, and the faculty of **Yenepoya Institute of Arts, Science, Commerce, and Management**. Their insights and assistance have been instrumental in successfully completing this internship.

Intern Signature:

Name: Joel Abin

Date: 15/05/2026

Institution Guide Signature:

Name: Munisah Muneer

Institute: Yenepoya Institute of Arts,
Science, Commerce, and Management

Date:

IBM Supervisor Signature:

**Head of the Department
(HOD) Signature:**

Name: Sumith Kumar Shukla

Designation: IBM Program Head

Organization: IBM

Date:

Name: Dr. Rathnakar Shetty P

Head of Department of Computer
Science

Institute: Yenepoya Institute of Arts,
Science, Commerce, and Management.

Date:

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Yenepoya Institute of Arts, Science, Commerce, and Management, affiliated with Yenepoya(Deemed to be University), Mangalore** for providing me with the opportunity to undertake this internship as part of my academic curriculum.

I extend my heartfelt thanks to **Dr. Jeevan Raj, Principal**, for his continuous support and guidance in facilitating this learning opportunity. I also express my sincere appreciation to **Dr. Rathnakara Shetty P**, Head of Department (Computer Science), for his encouragement and academic support throughout my internship journey.

I am deeply grateful to **Mr. Sumith Kumar Shukla**, for his visionary leadership and for fostering a dynamic learning environment. A special note of appreciation goes to **Mr. Shashank**, for his invaluable mentorship, expert guidance, and for sharing his vast experience in the field.

I am also thankful to my internal guide, **Ms. Munisah Muneer**, for her academic mentorship and for providing valuable insights that helped bridge the gap between theoretical learning and practical application.

Lastly, I am grateful to my faculty mentors, family, and peers for their encouragement and motivation throughout this journey.

Name: Joel Abin
Reg NO: 23BCCE108
Date: 15/05/2026

Yenepoya Project Tracker - 2026

Task ID	Task Description	Project Title	Project Timeline	Start Date	End Date	Status	Feedback and Comments
1	Configure project environment, install dependencies, and set up GitHub repository.	Cloud Forensics Automation for Retail Breaches	Week 1	21-03-2026	27-03-2026	Completed	Environment setup done. All tools installed and repository created successfully.
2	Develop retail transaction simulator and generate sample data.	Cloud Forensics Automation for Retail Breaches	Week 2	28-03-2026	03-04-2026	Completed	Transaction simulator working perfectly. Sample data generated.
3	Implement file monitoring, SHA-256 hashing, and tampering detection engine.	Cloud Forensics Automation for Retail Breaches	Week 3	04-04-2026	10-04-2026	Completed	Hashing and tampering detection working accurately.
4	Build Streamlit dashboard for alerts, logs, and forensic reports.	Cloud Forensics Automation for Retail Breaches	Week 4	11-04-2026	17-04-2026	Completed	Dashboard completed with all features and visualizations.
5	Test the complete system, generate reports, and deploy on GitHub.	Cloud Forensics Automation for Retail Breaches	Week 5	18-04-2026	24 – 04-2026	Completed	System fully tested and project uploaded to GitHub successfully.

ABSTRACT

Cloud Forensics Automation for Retail Breaches is a comprehensive cybersecurity and digital forensics platform developed to detect unauthorized modifications, suspicious activities, and integrity violations within retail transaction systems. Designed as an academic cybersecurity project, the platform integrates a simulated retail management application with an automated forensic monitoring engine to demonstrate practical security monitoring and incident detection in modern retail environments.

The system is developed using Python and Streamlit and operates on a lightweight file-based architecture that continuously monitors transaction logs, activity records, and critical retail data stored within the monitored environment. Instead of treating security as a secondary component, the project embeds forensic analysis directly into the operational workflow by implementing SHA-256 cryptographic hashing for integrity verification and automated tampering detection mechanisms. The platform continuously compares file hashes to identify unauthorized modifications and immediately flags altered files as potential security threats.

To simulate real-world retail breach scenarios, the system includes a live retail transaction simulator where sales records, inventory updates, and activity logs are dynamically generated. A forensic dashboard then performs automated scans on these files, extracts metadata, analyzes timestamps, and identifies suspicious activities such as malicious entries or unauthorized data manipulation. The platform also maintains audit logs and provides downloadable forensic reports in CSV format to preserve the chain of custody during investigations.

The application features an interactive Streamlit-based Security Operations Center (SOC) dashboard that visualizes system status, suspicious files, tampered records, and overall file integrity in real time. Additional functionalities include simulated cyberattack injection, automated report generation, and role-based monitoring workflows that help demonstrate incident response procedures in retail systems. Ultimately, Cloud Forensics Automation for Retail Breaches serves as a secure, practical, and user-friendly demonstration of how digital forensic techniques, file integrity monitoring, and automated threat detection can be integrated into retail infrastructures to improve cybersecurity resilience and support breach investigations.

TABLE OF CONTENTS

Background 1.....	1-2
1.1 Aim	2-3
1.2 Technologies.....	3-6
1.3 Hardware Architecture	6-10
1.4 SoftwareArchitecture	10-14
2 System Analysis.....	14-15
2.1 Existing System	15-16
2.2 Proposed System.....	17-19
2.3 Functional Requirements.....	19-22
2.4 Non-Functional Requirements.....	22-25
2.5 Feasibility Study	26-28
2.6 Methodology.....	28-31
2.7 System Design	32-35
2.8 Testing.....	35-38
2.9 Advantages of Proposed System.....	38-41
3 Implementation.....	41-45
4 Graphical User Interface (GUI).....	46-49
5 Results and Discussion.....	49-52
6 Conclusion.....	53-54
7 Future Scope.....	54-57
8 References.....	57
9 Appendix.....	57-60

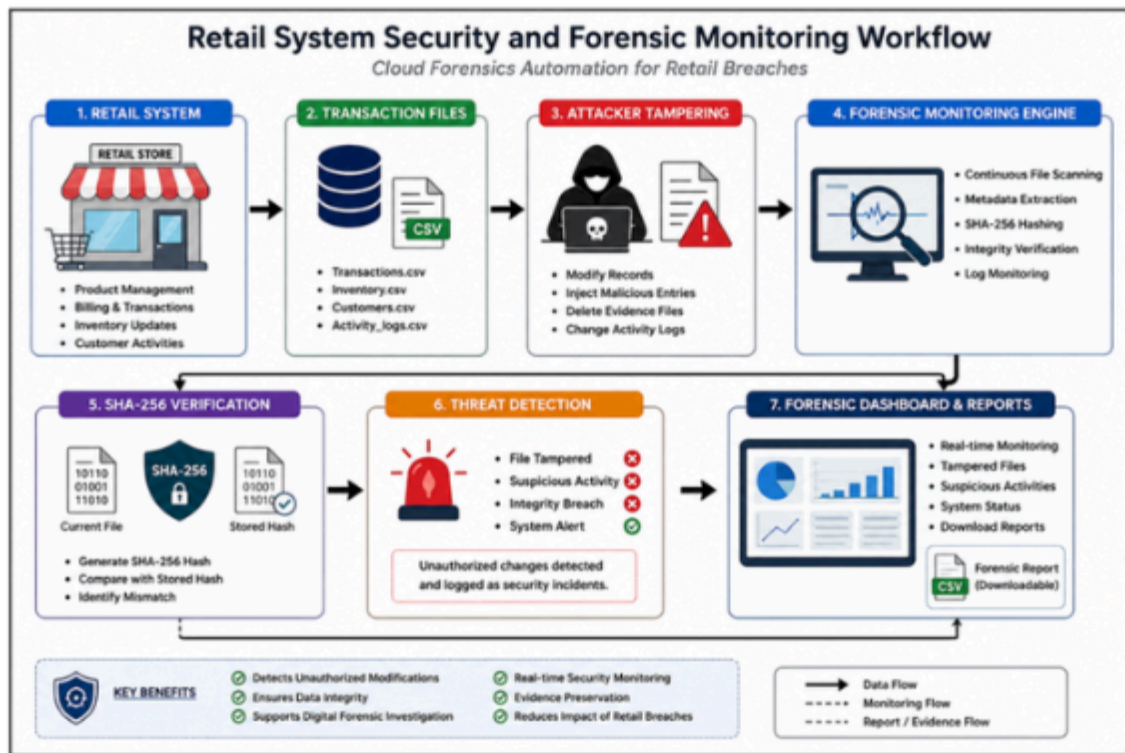
1. Background

The retail industry has rapidly transformed from traditional offline operations to highly digital and cloud-connected systems. Modern retail organizations now depend heavily on online billing systems, cloud storage, inventory databases, transaction servers, digital payment gateways, customer management systems, and centralized administrative platforms. Every transaction performed in a retail environment generates large volumes of digital data including customer details, billing information, inventory updates, payment records, and activity logs.

As retail infrastructures become increasingly dependent on digital technologies, they also become major targets for cybercriminals. Attackers may attempt to modify transaction records, delete important files, manipulate billing information, inject malicious entries, or tamper with activity logs in order to hide fraudulent activities. Retail breaches can result in financial loss, leakage of customer information, corruption of sales records, operational disruption, and reputational damage to organizations.

One of the major challenges in traditional retail systems is the lack of automated forensic monitoring and integrity verification mechanisms. Many retail platforms store transaction data without continuously validating whether files have been modified or tampered with. If an attacker gains access to the system and changes transaction records or log files, the organization may not immediately detect the attack. This delay in detection increases the risk of data compromise and makes forensic investigation more difficult.

Digital forensics plays a critical role in identifying, analyzing, and preserving evidence related to cyber incidents. File integrity monitoring and forensic analysis techniques help investigators determine whether important files have been altered, deleted, or manipulated. Cryptographic hashing algorithms such as SHA-256 are widely used in cybersecurity because they generate unique digital fingerprints for files. Even a small change in file content produces a completely different hash value, allowing investigators to identify unauthorized modifications quickly and accurately. The project “Cloud Forensics Automation for Retail Breaches” is developed to address these security challenges by integrating digital forensic analysis directly into a simulated retail environment. The system combines a live retail transaction platform with an automated forensic monitoring engine capable of continuously analyzing files stored inside the monitored environment.



1.1 Aim

The main aim of the project “Cloud Forensics Automation for Retail Breaches” is to design and develop a secure digital forensic monitoring system capable of detecting unauthorized modifications, suspicious activities, and integrity violations within retail transaction environments. The project focuses on integrating cybersecurity monitoring and forensic analysis techniques into a simulated retail infrastructure to improve security visibility and support cyber incident investigations.

The system is designed to continuously monitor retail transaction files, activity logs, inventory records, and operational data stored within monitored directories. By using SHA-256 cryptographic hashing, the platform verifies file integrity and identifies any unauthorized modifications performed on critical retail data. Whenever file tampering occurs, the system automatically flags the affected files and displays alerts through a centralized forensic monitoring dashboard.

Another important aim of the project is to demonstrate how digital forensic techniques can be automated for real-time cybersecurity monitoring. The platform performs continuous file analysis, metadata extraction, integrity verification, suspicious activity detection, and forensic report generation without requiring manual investigation for every operation. This helps improve the efficiency of identifying cyber threats and preserving digital evidence.

The project also aims to simulate real-world retail breach scenarios by creating a live retail transaction environment where customer purchases, billing operations, and inventory activities are dynamically generated. Attack simulations and malicious data injections are introduced into the monitored environment to demonstrate how cybercriminals may manipulate retail records and how forensic systems can detect such attacks effectively.

In addition, the project aims to provide a practical educational model for understanding the relationship between cloud security, digital forensics, file integrity monitoring, and incident response. The Streamlit-based Security Operations Center (SOC) dashboard enables investigators to visualize system status, suspicious files, tampered records, and forensic evidence in real time. The platform also generates downloadable CSV forensic reports that help preserve evidence and support further investigation processes.

Overall, the project aims to demonstrate how automated forensic monitoring systems can improve cybersecurity resilience, detect unauthorized modifications, preserve digital evidence, and strengthen the security of modern retail infrastructures against cyber threats and retail data breaches.

1.2 Technologies

The project “Cloud Forensics Automation for Retail Breaches” is developed using multiple technologies and tools that support cybersecurity monitoring, digital forensic analysis, dashboard visualization, and retail transaction simulation. The selected technologies provide a lightweight, scalable, and efficient environment for implementing real-time forensic monitoring and integrity verification within retail systems.

Python

Python is used as the core programming language for the entire project. It is selected because of its simplicity, flexibility, extensive library support, and strong capabilities in cybersecurity, automation, and file handling. Python is used to implement the forensic monitoring engine, file scanning operations, SHA-256 hashing mechanisms, transaction processing, report generation, and attack simulation logic.

- The project uses Python modules such as:
- `os` for file and directory handling
- `hashlib` for SHA-256 cryptographic hashing
- `csv` for forensic report generation
- `json` for storing hash records

- datetime for timestamp analysis
- pandas for data visualization and analysis

Python provides an efficient environment for automating digital forensic tasks and implementing cybersecurity workflows.

The system is constructed using the following core technologies, each chosen for its specific role in the secure application lifecycle:

Streamlit

Streamlit is used to develop the real-time forensic dashboard and Security Operations Center (SOC) interface. It enables rapid development of interactive web applications directly using Python code without requiring complex frontend frameworks.

The Streamlit dashboard provides:

- Real-time monitoring of files
- Visualization of suspicious activities
- Tampered file detection
- Dashboard metrics and statistics
- Downloadable forensic reports
- Attack simulation controls

The framework simplifies dashboard creation and allows investigators to interact with forensic data easily.

Flask

Flask is used to build the simulated retail transaction application. It acts as the backend server for generating customer purchases, billing records, inventory updates, and transaction logs within the retail environment.

The Flask-based retail system helps simulate:

- Product management
- Sales recording
- Customer transactions
- Inventory operations
- Activity logging

These generated files become the evidence source for forensic monitoring and integrity verification.

SHA-256 Hashing Algorithm

SHA-256 is a secure cryptographic hashing algorithm used for file integrity verification. The forensic engine calculates SHA-256 hash values for all monitored files and compares them with previously stored hash values to identify unauthorized modifications.

The hashing mechanism helps:

- Detect tampered files
- Verify file integrity
- Preserve digital evidence

- Identify unauthorized changes

Even a small modification in file content generates a completely different hash value, making SHA-256 highly effective for forensic analysis.

CSV Files

CSV (Comma-Separated Values) files are used for storing transaction records, activity logs, and downloadable forensic investigation reports. CSV files are lightweight, easy to process, and compatible with multiple data analysis tools.

The project generates:

- Transaction logs
- Inventory records
- Activity reports
- Forensic evidence reports

CSV-based reporting improves portability and simplifies forensic evidence management.

JSON

JSON (JavaScript Object Notation) is used to store SHA-256 hash values and integrity-related information. The hash database is maintained using JSON files because of their lightweight structure and easy readability.

JSON storage helps:

- Maintain stored hash records
- Compare integrity values
- Track file verification history

HTML and CSS

HTML and CSS are used for designing the frontend components of the retail transaction application. These technologies help create simple user interfaces for transaction entry and retail management operations.

The frontend includes:

- Product entry forms
- Transaction recording forms
- Transaction display tables
- Retail monitoring interface

Git and GitHub

Git is used for version control and project management. GitHub is used to host the project repository and maintain source code versions.

Git and GitHub provide:

- Source code backup
- Version tracking
- Collaboration support
- Project documentation management

The project repository is maintained online for academic review and future enhancements.

Operating System

The project is developed and tested on the Windows operating system. Windows provides the required environment for running Python, Streamlit, Flask, and forensic monitoring scripts efficiently.

Overall Technology Integration

The integration of Python, Streamlit, Flask, SHA-256 hashing, CSV reporting, and file monitoring technologies enables the development of a complete cloud forensic monitoring system for retail breach detection. These technologies work together to automate forensic analysis, visualize security incidents, detect unauthorized modifications, and preserve digital evidence within a simulated retail infrastructure.

1.3 Hardware Architecture

The hardware architecture of the project “Cloud Forensics Automation for Retail Breaches” is designed to support real-time forensic monitoring, retail transaction simulation, integrity verification, and dashboard visualization within a lightweight computing environment. The architecture demonstrates how digital forensic systems can operate within retail infrastructures to monitor files, analyze suspicious activities, and detect unauthorized modifications.

The system follows a centralized monitoring architecture where all retail transaction records, activity logs, inventory files, and operational data are stored inside monitored directories. The forensic monitoring engine continuously scans these files and performs integrity verification using SHA-256 cryptographic hashing. The generated forensic results are displayed through the Streamlit-based Security Operations Center (SOC) dashboard.

The hardware architecture mainly consists of the following components:

Client System

The client system represents the retail user environment where retail operations such as product management, billing, customer purchases, and transaction recording are performed. Users interact with the retail application through a web-based interface developed using Flask.

The client system is responsible for:

- Recording customer transactions
- Managing product details
- Updating inventory information
- Generating transaction logs
- Simulating retail operations

Every operation performed within the retail environment generates data that is stored for forensic monitoring and analysis.

Retail Transaction Server

The retail transaction server acts as the primary operational system of the project. It hosts the Flask-based retail application and processes all customer and inventory activities. The server generates transaction files, activity logs, and operational records that become digital evidence for forensic investigation.

The server performs:

- Transaction processing
- Data storage
- Activity logging
- Retail workflow execution
- File generation

The generated files are continuously monitored by the forensic engine to detect integrity violations.

Monitored Storage Environment

The monitored storage environment contains all important files related to retail operations. This includes:

- Transaction CSV files
- Inventory records
- Activity logs
- Suspicious access logs
- Backup files
- Configuration files

The forensic engine continuously scans this storage location to verify file integrity and identify unauthorized modifications.

This storage area acts as the primary evidence source for forensic analysis.

Forensic Monitoring Engine

The forensic monitoring engine is the core cybersecurity component of the project. It is developed using Python and performs automated digital forensic operations on the monitored files.

The engine performs:

- Continuous file scanning
- SHA-256 hash generation
- Metadata extraction
- File integrity verification
- Tampering detection
- Suspicious activity identification
- Report generation

Whenever file content changes, the engine compares the current hash value with the previously stored hash value. If a mismatch is detected, the file is marked as tampered.

Streamlit Dashboard System

The Streamlit dashboard functions as the Security Operations Center (SOC) interface of the project. It provides investigators with real-time visualization of system security status and forensic monitoring results.

The dashboard displays:

- Total monitored files
- Tampered files
- Suspicious activities

- Threat alerts
- File status distribution
- Downloadable forensic reports

The dashboard allows investigators to perform forensic scans and simulate attacks directly through the interface.

Local Database and Hash Storage

The project uses lightweight local storage mechanisms for maintaining:

- SHA-256 hash values
- Activity logs
- Report files
- Integrity verification records

JSON files are used to store hash values, while CSV files are used for evidence reporting and transaction storage.

This lightweight architecture simplifies deployment while supporting forensic analysis operations efficiently.

Network Connectivity

The project operates within a local host environment but simulates cloud-based monitoring workflows. Network connectivity is mainly used for:

- Dashboard access
- GitHub integration
- Cloud simulation
- Report sharing

The system architecture can later be expanded into cloud-based deployment environments for enterprise-scale retail monitoring.

Hardware Requirements

The minimum hardware requirements for running the project are:

Component Requirement

Processor

Intel i3/i5 or higher

RAM

Minimum 4 GB

Storage

Minimum 20 GB free space

Operating System

Windows 10/11

Internet Connection

Required for GitHub and cloud integration

Architecture Workflow

The overall workflow of the hardware architecture is as follows:

- Retail transactions are generated through the retail application.

- Transaction files and logs are stored inside monitored directories.
- The forensic monitoring engine continuously scans the stored files.
- SHA-256 hashes are generated and compared with stored values.
- Unauthorized modifications are identified as tampering incidents.
- The Streamlit dashboard displays alerts and forensic statistics.
- Investigation reports are generated and downloaded for analysis.

The hardware architecture successfully demonstrates how forensic monitoring systems can be integrated into retail infrastructures to improve cybersecurity visibility, detect integrity violations, and preserve digital evidence during cyber incidents.

1.4 Software Architecture

The software architecture of the project “Cloud Forensics Automation for Retail Breaches” is designed to integrate retail transaction processing, digital forensic monitoring, integrity verification, and real-time dashboard visualization into a single secure platform. The architecture follows a modular design approach where multiple software components work together to automate forensic analysis and detect unauthorized modifications within retail environments.

The system is primarily developed using Python and combines technologies such as Flask, Streamlit, SHA-256 hashing, CSV report generation, and file monitoring techniques. Each module within the architecture performs a dedicated function such as transaction generation, forensic scanning, integrity verification, activity logging, dashboard visualization, and attack simulation. The modular architecture improves maintainability, scalability, flexibility, and system performance.

The software architecture is divided into several important modules that collectively perform cybersecurity monitoring and digital forensic analysis operations.

Retail Management Module

The Retail Management Module acts as the transaction generation environment of the project. This module is developed using the Flask framework and simulates real-world retail operations such as product management, customer purchases, billing, and inventory updates. The module allows users to:

- Add products
- Record customer transactions
- Generate billing information
- Update inventory details

- Maintain transaction records

Every transaction generated within the retail environment is stored inside monitored files such as transaction CSV files and activity logs. These files later become digital evidence for forensic analysis and integrity verification.

The Retail Management Module plays an important role in simulating how real retail infrastructures continuously generate operational data that must be monitored and protected against unauthorized modifications.

File Monitoring Module

The File Monitoring Module continuously scans and monitors all files stored inside the monitored directories. The module is responsible for identifying file changes, collecting metadata information, and forwarding files for integrity verification.

The monitoring process includes:

- File scanning
- Directory traversal
- File access monitoring extraction
- Metadata
- Timestamp analysis

The module extracts important information such as:

- File name
- File path
- File size
- Creation time
- Modification time
- Access time

This information helps investigators analyze suspicious activities and understand when a file was modified or accessed.

The File Monitoring Module acts as the core evidence collection component of the system.

SHA-256 Integrity Verification Module

The SHA-256 Integrity VerificationModule is one of the most important cybersecurity components of the project. This module generates cryptographic SHA-256 hash values for all monitored files.

A SHA-256 hash acts as a unique digital fingerprint for a file. Even a very small change in file content produces a completely different hash value. The generated hash values are stored inside JSON-based hash storage files for future comparison.

The module performs:

- Hash generation Integrity
- verification Hash
- comparison Tampering
- identification

Whenever a forensic scan is performed, the current hash value of a file is compared with its previously stored hash value. If both values are different, the system identifies the file as tampered.

This mechanism ensures accurate file integrity verification and helps detect unauthorized modifications efficiently.

Tampering Detection Module

The Tampering Detection Module analyzes integrity verification results to identify suspicious modifications within the monitored environment. This module is responsible for detecting cyber incidents related to file manipulation and unauthorized data changes.

The module identifies:

- Hash mismatches
- Unauthorized file modifications
- Suspicious transaction updates
- Malicious data injections
- Integrity violations

Whenever tampering is detected, the affected files are flagged as suspicious and displayed on the forensic dashboard with warning alerts.

The Tampering Detection Module improves cybersecurity visibility and helps investigators identify compromised files quickly.

Attack Simulation Module

The Attack Simulation Module is developed to demonstrate real-world cyberattack scenarios within retail systems. This module allows simulated malicious modifications to be inserted into transaction records and activity logs.

The attack simulation process helps:

- Demonstrate file tampering
- Validate forensic monitoring logic
- Test integrity verification
- Simulate insider threats
- Simulate retail data breaches

The module provides practical understanding of how attackers manipulate digital evidence and how forensic systems identify integrity violations using cryptographic analysis.

Streamlit Dashboard Module

The Streamlit Dashboard Module acts as the Security Operations Center (SOC) interface of the project. It provides real-time visualization of forensic monitoring results and cybersecurity alerts.

The dashboard displays:

- Total monitored files
- Tampered files
- Suspicious activities
- Threat alerts
- File integrity statistics
- Investigation reports

The dashboard also allows investigators to:

- Run forensic scans
- Simulate attacks
- Monitor activity logs
- Download CSV reports
- Analyze file integrity status

The dashboard simplifies forensic monitoring and improves accessibility for investigators and administrators.

Report Generation Module

The Report Generation Module generates forensic investigation reports in CSV format. The reports contain detailed information related to monitored files and integrity verification results.

The generated reports include:

- File names
- File paths
- SHA-256 hash values
- File timestamps
- Integrity status
- Suspicious activity indicators

The reports help preserve digital evidence and support forensic investigations.

Activity Logging Module

The Activity Logging Module records important system activities and monitoring operations. The logs help maintain accountability and preserve operational evidence.

The logging module records:

- User activities
- File scan operations
- Dashboard access
- Attack simulations
- Tampering alerts

These logs help investigators analyze system events during cyber investigations.

Overall Workflow of Software Architecture

The complete software architecture workflow operates as follows:

- The retail application generates transaction records and activity logs.
- Files are stored inside monitored directories.
- The forensic monitoring engine continuously scans the files.
- SHA-256 hash values are generated for integrity verification.

- Current hashes are compared with previously stored values.
- Unauthorized modifications are identified as tampering incidents.
- The dashboard displays real-time alerts and forensic statistics.
- Investigation reports are generated for evidence analysis.

The software architecture successfully integrates cybersecurity monitoring, digital forensics, integrity verification, attack simulation, and real-time dashboard visualization into a single unified platform capable of detecting retail breaches and unauthorized data modifications effectively.

2. System Analysis

System analysis is an important phase in software development that focuses on studying the existing system, identifying its limitations, understanding security challenges, and developing an improved solution to overcome those problems. In cybersecurity-based applications, system analysis plays a major role in identifying vulnerabilities, weak monitoring mechanisms, and integrity risks that may lead to cyberattacks or unauthorized data modifications.

Modern retail systems continuously generate large amounts of digital information such as transaction records, customer billing details, inventory data, activity logs, and operational files. These files contain sensitive information that must be protected against unauthorized access, tampering, and cyber threats. However, many traditional retail systems focus mainly on operational efficiency and billing management while providing limited support for forensic monitoring and integrity verification.

One of the major problems in existing retail environments is the lack of automated cybersecurity monitoring systems capable of detecting unauthorized modifications in real time. If attackers modify transaction records, inject malicious entries, delete important logs, or manipulate billing information, the organization may not immediately detect the attack. Delayed detection increases the difficulty of forensic investigation and creates major risks for retail infrastructures.

The project “Cloud Forensics Automation for Retail Breaches” is developed after analyzing these security limitations within traditional retail systems. The project introduces an automated forensic monitoring platform capable of continuously analyzing transaction files and detecting unauthorized modifications using SHA-256 cryptographic hashing and integrity verification mechanisms.

The system analysis phase focuses on:

- Identifying weaknesses in existing retail systems
- Understanding forensic monitoring requirements
- Studying integrity verification mechanisms
- Designing automated tampering detection
- Improving cybersecurity visibility
- Supporting forensic evidence preservation
- Developing real-time monitoring solutions

The proposed system integrates retail transaction simulation with digital forensic monitoring to create a secure environment capable of identifying suspicious activities and integrity violations efficiently. The system combines file monitoring, cryptographic hashing, dashboard visualization, attack simulation, and report generation into a single cybersecurity platform designed for retail breach analysis and forensic investigation

2.1 Existing System

The existing retail systems used in many organizations are primarily designed to manage sales operations, billing processes, inventory records, and customer transactions. These systems focus mainly on improving business efficiency, transaction speed, and operational management. However, most traditional retail systems provide very limited cybersecurity monitoring and forensic investigation capabilities.

In many retail environments, transaction records and activity logs are stored locally or within cloud storage systems without continuous integrity verification. If an attacker gains unauthorized access to the system and modifies transaction data, deletes activity logs, or injects malicious records, the changes may remain undetected for long periods of time. This creates serious security risks and increases the difficulty of identifying cyber incidents during forensic investigations.

Traditional retail systems mainly depend on manual monitoring and administrator-based investigation methods. Administrators usually review logs and transaction records only after suspicious activities are reported. Manual forensic investigation consumes significant time and may delay incident response processes. In many cases, attackers modify important files or remove evidence before the organization identifies the breach. Most existing systems also lack automated integrity verification mechanisms such as cryptographic hashing. Without integrity verification, organizations cannot easily determine whether transaction records or operational files have been modified without authorization. Attackers may manipulate billing values, alter customer transaction histories, or inject fraudulent entries into activity logs without generating immediate alerts.

Another limitation of traditional retail infrastructures is the absence of centralized dashboard-based monitoring systems. Security-related information is often scattered across multiple logs and files, making it difficult for investigators to analyze incidents efficiently. The lack of real-time monitoring and automated threat detection reduces overall cybersecurity visibility within retail environments.

The existing retail systems also suffer from weak evidence preservation mechanisms. Since activity logs and transaction records are not continuously monitored or protected using integrity verification techniques, important forensic evidence may be lost or modified during cyber incidents. This creates challenges during digital forensic analysis and incident response procedures.

Problems in Existing System

The major problems identified in the existing retail systems are:

Lack of Automated Monitoring

Traditional systems do not continuously monitor files and transaction records for unauthorized changes.

No Integrity Verification

Most systems lack SHA-256 or cryptographic hashing mechanisms for verifying file integrity.

Manual Investigation Process

Incident investigations are performed manually, increasing investigation time and reducing efficiency.

No Real-Time Threat Detection

Existing systems cannot immediately detect suspicious modifications or tampering activities.

Weak Security Visibility

The absence of centralized dashboards makes monitoring and forensic analysis difficult.

Poor Evidence Preservation

Transaction records and logs are vulnerable to unauthorized modifications and deletion.

Increased Risk of Data Tampering

Attackers can manipulate retail records without generating automated alerts.

Limitations of Existing System

Existing System Limitations			Impact
	No file integrity monitoring	→	Tampering remains undetected
	Lack of forensic automation	→	Slow investigation process
	Manual log analysis	→	Increased operational effort
	No centralized dashboard	→	Poor threat visibility
	Weak cybersecurity monitoring	→	Increased attack risk
	No automated alerts	→	Delayed incident response
	No evidence verification	→	Difficult forensic analysis

2.2 Proposed System

The proposed system “Cloud Forensics Automation for Retail Breaches” is designed to overcome the limitations and security weaknesses present in traditional retail systems. The system integrates digital forensic monitoring, file integrity verification, tampering detection, real-time dashboard visualization, and forensic report generation into a single secure platform. The main objective of the proposed system is to automatically identify unauthorized modifications and suspicious activities within retail transaction environments.

The proposed system combines a simulated retail management application with an automated forensic monitoring engine. The retail application continuously generates transaction records, billing information, inventory updates, and activity logs that are stored inside monitored directories. The forensic engine continuously scans these files, analyzes their metadata, generates SHA-256 hash values, and compares them with previously stored values to verify file integrity.

Whenever a file is modified without authorization, the system immediately detects the integrity violation through hash mismatch analysis. The affected files are automatically marked as tampered, and alerts are displayed through the forensic dashboard. This enables investigators and administrators to quickly identify suspicious modifications and respond to cybersecurity incidents effectively.

The proposed system introduces real-time forensic monitoring capabilities that are absent in traditional retail systems. Instead of relying on manual investigation processes, the platform automates file analysis, integrity verification, threat detection, activity logging, and forensic report generation. This improves investigation speed, enhances cybersecurity visibility, and reduces operational effort.

The project also includes attack simulation functionality to demonstrate how cyberattacks affect retail transaction records. Simulated malicious entries can be injected into monitored files to imitate real-world tampering attacks. The forensic engine automatically identifies these modifications and updates the dashboard with threat alerts. This functionality helps validate the effectiveness of the monitoring system and provides practical understanding of digital forensic investigation techniques.

The proposed system is developed using Python, Flask, Streamlit, SHA-256 cryptographic hashing, CSV report generation, and file monitoring mechanisms. The Streamlit-based dashboard acts as a Security Operations Center (SOC) interface that provides centralized monitoring and real-time visualization of system security status.

The proposed system performs the following major operations:

- Retail transaction simulation
- Continuous file monitoring
- SHA-256 integrity verification
- Tampering detection Suspicious
- activity analysis Dashboard
- visualization
- Attack simulation
- Activity logging
- CSV forensic report generation

Features of Proposed System

Automated Forensic Monitoring

The system continuously monitors transaction files and operational records automatically without requiring manual investigation.

SHA-256 Integrity Verification

Cryptographic hashing is used to verify file integrity and detect unauthorized modifications accurately.

Real-Time Tampering Detection

The system immediately identifies suspicious modifications and displays alerts through the dashboard.

Centralized Dashboard Monitoring

The Streamlit dashboard provides centralized visibility of all monitoring activities and forensic alerts.

Attack Simulation

The system supports simulated cyberattacks to demonstrate tampering detection and forensic analysis.

Evidence Preservation

The system generates downloadable forensic reports and maintains activity logs for evidence analysis.

Improved Cybersecurity Visibility

Investigators can monitor suspicious activities and integrity violations in real time.

Advantages of Proposed System

Proposed System Features		Benefits
 Automated monitoring		Faster threat detection
 SHA-256 verification		Accurate integrity checking
 Real-time alerts		Immediate incident response
 Dashboard visualization		Better security visibility
 CSV report generation		Improved evidence preservation
 Activity logging		Better forensic traceability
 Attack simulation		Practical cybersecurity testing

Workflow of Proposed System

The proposed system operates through the following workflow:

- 1.The retail application generates transaction records and activity logs.
- 2.Files are stored inside monitored directories.
- 3.The forensic engine continuously scans all monitored files.
- 4.SHA-256 hash values are generated for integrity verification.
- 5.Current hash values are compared with previously stored values.
- 6.Unauthorized modifications are identified as tampering incidents.
- 7.The dashboard displays alerts and forensic monitoring statistics.
- 8.Investigation reports are generated in CSV format for evidence analysis.

The proposed system successfully integrates cybersecurity monitoring, digital forensic analysis, integrity verification, attack simulation, and dashboard visualization into a unified platform capable of detecting retail data breaches and unauthorized modifications efficiently.

2.3 Functional Requirements

Functional requirements describe the core operations and services that the system must perform to achieve the objectives of the project. In the project “Cloud Forensics Automation for Retail Breaches,” the functional requirements define how the system performs retail transaction simulation, forensic monitoring, integrity verification, tampering detection, dashboard visualization, and forensic report generation.

The functional requirements are designed to ensure that the platform can continuously monitor retail transaction files, identify unauthorized modifications, preserve digital evidence, and provide real-time cybersecurity visibility through the forensic dashboard.

User Authentication Requirement

The system must provide secure login access for authorized users. Only authenticated users should be allowed to access the forensic dashboard and monitoring functionalities.

The authentication system should:

- Verify username and password
- Prevent unauthorized access
- Maintain secure user sessions
- Protect investigation data

This functionality improves system security and protects sensitive forensic information.

Retail Transaction Management Requirement

The system must provide a simulated retail environment capable of generating transaction records and operational files for forensic monitoring.

The retail module should:

- Add products
- Record customer purchases
- Generate transaction logs
- Maintain sales records
- Update inventory information

These generated files become evidence sources for forensic analysis.

File Monitoring Requirement

The system must continuously monitor files stored inside the monitored directories.

The monitoring engine should:

- Scan files automatically
- Detect file modifications
- Identify newly added files
- Analyze file metadata
- Monitor file timestamps

This requirement supports evidence collection and suspicious activity detection.

SHA-256 Integrity Verification Requirement

The system must generate SHA-256 hash values for all monitored files to verify file integrity.

The hashing module should:

- Generate unique digital fingerprints
- Store hash values securely

- Compare old and new hash values
- Detect integrity violations

This mechanism helps identify unauthorized modifications accurately.

Tampering Detection Requirement

The system must identify unauthorized modifications using hash comparison mechanisms.

The tampering detection module should:

- Detect hash mismatches
- Identify suspicious modifications
- Flag compromised files
- Generate tampering alerts

This functionality helps investigators quickly identify cyber incidents.

Dashboard Visualization Requirement

The system must provide a centralized dashboard for monitoring forensic activities and cybersecurity alerts.

The dashboard should display:

- Total monitored files
- Tampered files
- Suspicious files
- Threat alerts
- File integrity statistics
- Investigation logs

This functionality improves security visibility and monitoring efficiency.

AttackSimulation Requirement

The system must support attack simulation features for demonstrating tampering detection.

The attack simulation module should:

- Inject malicious entries
- Modify transaction files
- Trigger forensic alerts
- Validate integrity verification

This functionality demonstrates how forensic systems identify cyberattacks.

CSV Report Generation Requirement

The system must generate downloadable forensic reports in CSV format.

The reports should contain:

- File names
- File paths
- SHA-256 hash values
- File timestamps
- Integrity status
- Suspicious activity indicators

The reports help preserve digital evidence for investigation purposes.

Activity Logging Requirement

The system must maintain activity logs for all important operations performed within the environment.

The logging module should record:

- Login activities
- File scans
- Tampering incidents
- Dashboard operations
- Attack simulations

These logs improve forensic traceability and accountability.

Functional Workflow of the System

The overall workflow of the system operates as follows:

- Retail transactions are generated through the retail application.
- Files are stored inside monitored directories.
- The forensic engine continuously scans the files.
- SHA-256 hashes are generated and verified.
- Unauthorized modifications are detected through hash comparison.
- Tampering alerts are displayed on the dashboard.
- Reports are generated for forensic analysis and evidence preservation.

The functional requirements collectively ensure that the system performs automated forensic monitoring, integrity verification, cybersecurity visualization, and tampering detection effectively within a retail environment.

2.4 Non-Functional Requirements

Non-functional requirements define the quality attributes, operational standards, and performance characteristics of the system. These requirements focus on how efficiently, securely, reliably, and accurately the system performs its operations. In the project “Cloud Forensics Automation for Retail Breaches,” the non-functional requirements ensure that the forensic monitoring platform operates securely and efficiently while supporting continuous cybersecurity analysis and evidence preservation.

The non-functional requirements are important because digital forensic systems must maintain data integrity, system reliability, monitoring accuracy, and real-time visibility without affecting overall system performance.

Security Requirement

Security is one of the most important non-functional requirements of the project. Since the system handles sensitive transaction records and forensic evidence, it must protect files against unauthorized access and tampering.

The system should:

- Restrict unauthorized access
- Protect forensic reports
- Prevent unauthorized file modifications

- Secure monitoring operations
- Maintain integrity verification using SHA-256 hashing

Security mechanisms help preserve digital evidence and improve cyberattack detection capabilities.

Reliability Requirement

The system must operate reliably without interruption during monitoring and forensic analysis operations.

The system should:

- Continuously monitor files
- Perform accurate forensic scans
- Generate reliable integrity verification results
- Maintain stable dashboard functionality

Reliable operation is important because missed detections may allow unauthorized modifications to remain unnoticed.

Performance Requirement

The system must perform file monitoring and integrity verification efficiently without causing significant delays.

The monitoring engine should:

- Scan files quickly
- Generate SHA-256 hashes efficiently
- Detect modifications in real time
- Update dashboard alerts without delay

Efficient performance improves investigation speed and enhances real-time monitoring capabilities.

Scalability Requirement

The system should support future expansion and integration of additional cybersecurity functionalities.

The architecture should allow:

- Monitoring of additional directories
- Integration with cloud storage systems
- Support for larger transaction datasets
- Future AI-based threat detection modules

Scalability ensures long-term usability of the system.

Availability Requirement

The forensic monitoring platform should remain available whenever monitoring operations are required.

The system should:

- Support continuous monitoring
- Maintain dashboard accessibility

- Ensure uninterrupted scan operations
- Preserve monitoring logs during execution

High availability improves incident response efficiency.

Usability Requirement

The system should provide a simple and user-friendly interface for investigators and administrators.

The dashboard should:

- Display alerts clearly
- Provide easy navigation
- Simplify forensic analysis
- Allow quick report downloads

Improved usability helps users monitor cybersecurity incidents efficiently without requiring advanced technical expertise.

Maintainability Requirement

The system should be easy to maintain, modify, and upgrade.

The project architecture should:

- Support modular development
- Simplify debugging
- Allow easy feature updates
- Improve code readability

Maintainability reduces development complexity and supports future enhancements.

Portability Requirement

The system should be capable of running on multiple configuration environments with minimal changes.

The platform should support:

- Windows operating systems
- Python environments
- Local execution
- Cloud deployment possibilities

Portability improves deployment flexibility.

Integrity Requirement

The system must maintain accurate integrity verification for all monitored files.

The forensic engine should:

- Generate accurate hash values
- Detect unauthorized modifications
- Preserve evidence consistency
- Prevent integrity mismatches

Integrity verification is essential for reliable forensic investigations.

Logging Requirement

The system should maintain detailed activity logs for all important operations.

The logs should record:

- File scan operations
- Dashboard activities
- Attack simulations
- Tampering incidents
- User actions

Logging improves accountability and forensic traceability.

Non-Functional Requirement Summary

Non-Functional Requirement Summary		
Non-Functional Requirement		Purpose
	Security	Protect forensic data and monitoring operations
	Reliability	Ensure stable monitoring and detection
	Performance	Improve scan and detection speed
	Scalability	Support future expansion
	Availability	Maintain continuous operation
	Usability	Improve user interaction
	Maintainability	Simplify updates and debugging
	Portability	Support multiple environments
	Integrity	Ensure accurate verification
	Logging	Preserve operational evidence

The non-functional requirements collectively ensure that the system operates securely, reliably, efficiently, and accurately while supporting automated forensic monitoring and cybersecurity analysis within retail environments.

2.5 Feasibility Study

A feasibility study is an important part of system analysis that evaluates whether the proposed project can be successfully developed and implemented within the available resources, technologies, budget, and operational environment. The feasibility study helps determine whether the system is practical, achievable, and beneficial for solving the identified problem.

In the project “Cloud Forensics Automation for Retail Breaches,” the feasibility study analyzes the technical, economic, operational, and implementation aspects of the proposed forensic monitoring system. The study confirms that the system can be effectively developed using available technologies and deployed within a simulated retail environment for cybersecurity and digital forensic analysis.

The project is designed using lightweight technologies such as Python, Flask, Streamlit, CSV-based storage, and SHA-256 hashing algorithms, which makes the implementation practical and cost-effective for academic and research purposes.

Technical Feasibility

Technical feasibility evaluates whether the required technologies, software tools, and development resources are available for successful implementation of the system.

The proposed project is technically feasible because it uses widely available technologies and open-source software frameworks. The system is developed using Python programming language along with Flask and Streamlit frameworks, which are lightweight, efficient, and easy to integrate.

The project also uses:

- SHA-256 cryptographic hashing
- File monitoring mechanisms
- CSV report generation
- JSON-based hash storage
- Dashboard visualization tools

All these technologies are compatible with modern computer systems and do not require expensive hardware or specialized infrastructure.

The technical feasibility of the project is supported by:

- Availability of open-source tools
- Simple software architecture
- Easy development environment setup Compatibility with Windows systems
- Low hardware requirements
- Easy integration of cybersecurity functionalities

The project can be executed successfully on standard systems with moderate computing resources.

Economic Feasibility

Economic feasibility determines whether the project can be developed and maintained within reasonable cost limits.

The proposed system is economically feasible because it relies mainly on open-source technologies and free development frameworks. No expensive commercial software, enterprise licenses, or specialized hardware components are required for implementation.

The project minimizes development cost by using:

- Python open-source environment
- Flask framework
- Streamlit dashboard framework
- CSV-based storage
- Built-in hashing libraries

The hardware requirements are also minimal, which reduces infrastructure expenses. Since the project is developed mainly for academic and cybersecurity research purposes, operational costs remain very low.

The economic benefits of the project include:

- Low implementation cost
- No license expenses
- Minimal maintenance cost
- Reduced investigation effort
- Faster detection of suspicious activities

Therefore, the project is highly cost-effective.

Operational Feasibility

Operational feasibility evaluates whether the proposed system can operate effectively within the intended environment and whether users can utilize the system efficiently.

The proposed forensic monitoring system is operationally feasible because it provides a simple, user-friendly interface through the Streamlit dashboard. The dashboard allows users to:

- Monitor file integrity
- Run forensic scans
- Detect tampering
- Analyze suspicious activities
- Download investigation reports

The system does not require advanced technical expertise for basic monitoring operations, making it suitable for academic demonstrations and cybersecurity analysis.

The project also supports:

- Continuous monitoring
- Real-time alert generation
- Automated integrity verification
- Centralized dashboard visibility

These features improve operational efficiency and simplify forensic analysis processes.

Implementation Feasibility

Implementation feasibility analyzes whether the project can be successfully developed within the available time, resources, and development environment.

The proposed system is easy to implement because:

- The project uses lightweight frameworks
- The software architecture is modular
- Python simplifies backend development
- Streamlit simplifies dashboard creation
- SHA-256 hashing is directly supported through Python libraries

The project can be developed incrementally by implementing individual modules such as:

- Retail transaction module
- File monitoring module
- Integrity verification module
- Dashboard module
- Report generation module

This modular implementation approach simplifies testing, debugging, and deployment.

Feasibility Study Summary

Feasibility Type		Description
	Technical Feasibility	Uses available technologies and open-source tools
	Economic Feasibility	Low implementation and maintenance cost
	Operational Feasibility	Easy monitoring and dashboard usability
	Implementation Feasibility	Simple modular development process

The feasibility study confirms that the project “Cloud Forensics Automation for Retail Breaches” is practical, cost-effective, technically achievable, and operationally efficient for implementing automated forensic monitoring and cybersecurity analysis within retail environments.

2.6 Methodology

Methodology describes the overall process, workflow, and implementation approach followed during the development of the project. In the project “Cloud Forensics Automation for Retail Breaches,” the methodology focuses on integrating retail transaction simulation, automated forensic monitoring, integrity verification, tampering detection, dashboard visualization, and forensic report generation into a unified cybersecurity platform.

The methodology is designed to simulate a real-world retail environment where transaction records and operational files are continuously generated and monitored for suspicious activities. The project combines digital forensic analysis techniques with cybersecurity monitoring mechanisms to identify unauthorized modifications and preserved digital evidence efficiently.

The methodology of the project follows a modular and step-by-step implementation approach. Each module performs a dedicated function within the forensic monitoring workflow, allowing the system to operate efficiently and maintain accurate integrity verification.

The complete methodology consists of the following major phases:

Retail Transaction Generation Phase

The first phase of the methodology focuses on generating retail transaction data through the simulated retail application. The retail management system allows users to:

- Add products
- Record customer purchases
- Generate billing information
- Maintain inventory records
- Create transaction logs

Every operation performed within the retail application generates digital files such as transaction records and activity logs. These files are stored inside monitored directories and later analyzed by the forensic monitoring engine.

This phase simulates how real retail infrastructures continuously generate operational data that may become targets for cyberattacks and unauthorized modifications.

FileMonitoringPhase

The second phase involves continuous monitoring of files stored inside the monitored directories. The forensic monitoring engine scans all files and collects important metadata information required for investigation and integrity analysis.

The monitoring system performs:

- Directory scanning
- File access monitoring Metadata
- extraction Timestamp analysis
- File modification tracking

Important file details collected during this phase include:

- File name
- File path
- File size
- Modification time
- Access time
- Creation time

This phase acts as the evidence collection stage of the forensic workflow.

SHA-256Integrity Verification Phase

In this phase, the forensic engine generates SHA-256 cryptographic hash values for all monitored files. The generated hash values act as digital fingerprints that uniquely identify file contents.

The integrity verification process includes:

- Generating SHA-256 hashes
- Storing hash values securely
- Comparing current and previous hashes
- Verifying file consistency

If even a small modification occurs within a file, the SHA-256 hash value changes completely. This allows the system to identify unauthorized modifications accurately. This phase forms the core cybersecurity mechanism of the project.

Tampering DetectionPhase

The tampering detection phase identifies suspicious modifications using integrity verification results. The forensic engine compares current file hashes with previously stored hash values. If both values do not match:

- The file is marked as tampered
- Threat alerts are generated
- Dashboard status is updated
- Suspicious activities are recorded

The tampering detection mechanism helps investigators quickly identify integrity violations and suspicious activities within the monitored environment.

DashboardVisualization Phase

The dashboard visualization phase provides centralized monitoring and real-time cybersecurity visibility through the Streamlit dashboard interface.

The dashboard displays:

- Total monitored files
- Tampered files
- Suspicious activities
- Threat alerts
- File integrity statistics
- Monitoring reports

Investigators can:

- Run forensic scans
- Monitor security status Simulate
- attacks
- Analyze suspicious files
- Download investigation reports

This phase improves forensic monitoring accessibility and simplifies cyber incident analysis.

Attack SimulationPhase

The attack simulation phase demonstrates how cyberattacks affect retail transaction environments.

Simulated malicious entries and unauthorized modifications are injected into monitored files.

The attack simulation process helps:

- Validate forensic monitoring logic
- Demonstrate tampering detection
- Simulate insider attacks
- Test integrity verification
- Analyze cyberattack impact

The forensic engine automatically identifies these modifications and updates the dashboard with threat alerts.

Report Generation Phase

The final phase generates forensic investigation reports in CSV format. The generated reports preserve important evidence related to monitored files and suspicious activities.

The reports include:

- File names
- File paths
- SHA-256 hash values
- Integrity status
- Modification timestamps
- Suspicious activity indicators

The reports support digital forensic investigations and improve evidence preservation.

Overall Methodology Workflow

The complete workflow of the methodology operates as follows:

- Retail transactions are generated through the retail application.
- Files are stored inside monitored directories.
- The forensic monitoring engine continuously scans the files.
- SHA-256 hashes are generated for integrity verification.
- Current hashes are compared with previously stored values.
- Unauthorized modifications are detected.
- Threat alerts are displayed on the dashboard.
- Investigation reports are generated for evidence analysis.

The methodology successfully integrates digital forensic investigation techniques, integrity verification mechanisms, cybersecurity monitoring, attack simulation, and dashboard visualization into a practical platform capable of detecting retail breaches and unauthorized modifications effectively.

2.7 System Design

System design describes the structural organization, component interaction, workflow management, and implementation structure of the proposed forensic monitoring platform. It defines how different modules within the project communicate with each other to perform dashboard monitoring, integrity verification, tampering detection, cybersecurity visualization, and forensic report generation.

The system design of the project “Cloud Forensics Automation for Retail Breaches” follows a modular architecture where multiple software components operate together in a coordinated manner. The design ensures scalability, maintainability, efficient monitoring, and real-time forensic analysis within the simulated retail environment.

The project combines a retail transaction platform with a forensic monitoring engine to simulate real-world cyberattack scenarios and digital forensic investigations. Each module within the system performs a specific operation and contributes to the overall cybersecurity workflow.

The major components included in the system design are:

- Retail Management System
- File Monitoring Engine
- SHA-256 Integrity Verification System
- Tampering Detection Engine
- Dashboard Visualization Module
- Activity Logging System
- CSV Report Generation Module
- Attack Simulation Module

These components collectively form the complete forensic monitoring ecosystem.

Retail Management System Design

The Retail Management System acts as the primary transaction generation environment of the project. This module is responsible for simulating retail operations such as:

- Product management
- Customer transactions
- Inventory updates
- Billing processes
- Sales record generation

Whenever a transaction occurs, the system generates transaction records and activity logs that are stored inside monitored directories. These files later become evidence sources for forensic analysis.

The retail application is developed using Flask and operates through a lightweight web interface.

File Monitoring Engine Design

The File Monitoring Engine continuously scans all files stored inside the monitored directories. This module is responsible for identifying file modifications and collecting metadata information for forensic investigation.

The monitoring engine performs:

- Continuous file scanning
- File metadata collection
- Timestamp extraction
- Modification tracking
- Evidence collection

The monitoring system acts as the core forensic analysis component of the project.

SHA-256 Integrity Verification Design

The SHA-256 Integrity Verification module generates cryptographic hash values for all monitored files. These hash values are used to verify file integrity and identify unauthorized modifications.

The integrity verification workflow includes:

1. File selection
2. SHA-256 hash generation
3. Hash storage
4. Hash comparison
5. Integrity verification

If a file is modified, the generated hash value changes completely, allowing the system to identify tampering accurately.

Tampering Detection Engine Design

The Tampering Detection Engine analyzes integrity verification results to identify suspicious modifications and integrity violations.

The module performs:

- Hash mismatch detection
- Suspicious file identification
- Threat alert generation
- Dashboard updates

Whenever tampering is detected, the affected files are automatically marked as compromised and displayed through the dashboard interface.

Dashboard Visualization Design

The dashboard module acts as the Security Operations Center (SOC) interface of the project. It provides centralized monitoring and real-time visualization of cybersecurity activities.

The dashboard displays:

- Total monitored files
- Tampered files

- Threat alerts
- File integrity statistics
- Activity logs
- Investigation reports

The dashboard also provides user controls for:

- Running forensic scans
- Simulating attacks Downloading reports
- Monitoring suspicious activities

The dashboard is developed using Streamlit and supports simple user interaction.

Activity Logging Design

The Activity Logging Module records all important monitoring activities and operational events within the system.

The logs include:

- File scan operations
- Tampering incidents
- Dashboard activities
- Attack simulations
- User interactions

The logging system helps preserve forensic evidence and improves investigation traceability.

Report Generation Design

The Report Generation Module creates downloadable forensic reports in CSV format. The generated reports contain:

- File names
- File paths
- SHA-256 hash values Integrity
- status
- File timestamps
- Suspicious activity indicators

The reports help investigators analyze incidents and preserve digital evidence.

Attack Simulation Design

The Attack Simulation Module demonstrates how attackers may manipulate retail transaction files and operational records.

The module performs:

- Malicious entry injection
- File modification simulation
- Tampering trigger generation
- Dashboard alert activation

This functionality helps validate the effectiveness of the forensic monitoring engine.

Overall System Workflow

The complete system workflow operates through the following sequence:

1. Retail transactions are generated through the retail application.
2. Files are stored inside monitored directories.
3. The monitoring engine scans all files continuously.
4. SHA-256 hashes are generated and stored.
5. Current hashes are compared with previous values.
6. Tampering incidents are identified through integrity mismatches.
7. Dashboard alerts are generated for investigators.
8. Investigation reports are created and downloaded for forensic analysis.

The system design successfully integrates cybersecurity monitoring, digital forensic investigation, integrity verification, attack simulation, and dashboard visualization into a single unified platform capable of detecting unauthorized modifications and retail data breaches effectively.

2.8 Testing

Testing is an important phase in software development that verifies whether the system performs correctly, securely, and efficiently under different operational conditions. In the project “Cloud Forensics Automation for Retail Breaches,” testing is performed to validate forensic monitoring operations, integrity verification accuracy, tampering detection mechanisms, dashboard functionality, and report generation processes.

The testing phase ensures that the system successfully detects unauthorized modifications, preserves digital evidence, generates accurate forensic reports, and provides real-time cybersecurity visibility through the dashboard interface.

The project undergoes multiple testing procedures to verify the performance and reliability of each module within the forensic monitoring platform.

The major testing areas include:

- Functional testing
- File monitoring testing
- Integrity verification testing
- Tampering detection testing
- Dashboard testing
- Report generation testing
- Attack simulation testing

Each testing process validates a specific functionality of the system.

Functional Testing

Functional testing verifies whether all modules within the project operate according to the specified requirements.

The following functionalities were tested:

- Product addition
- Transaction recording

- File monitoring
- SHA-256 hash generation
- Dashboard visualization
- CSV report generation
- Attack simulation

The testing confirmed that all modules performed successfully without major operational errors.

File Monitoring Testing

File monitoring testing verifies whether the forensic engine continuously scans and analyzes files stored inside monitored directories.

The monitoring engine was tested for:

- File detection Modification
- tracking Metadata extraction
- Timestamp analysis

The testing confirmed that the system successfully identified newly added files and monitored file modifications accurately.

Integrity Verification Testing

Integrity verification testing validates the accuracy of the SHA-256 hashing mechanism.

The testing process included:

1. Generating initial hash values
2. Modifying monitored files
3. Regenerating hash values
4. Comparing old and new hashes

The testing confirmed that even small modifications produced completely different SHA-256 hash values, allowing accurate integrity verification.

Tampering Detection Testing

Tampering detection testing verifies whether the system correctly identifies unauthorized modifications within monitored files.

The testing process involved:

- Injecting malicious entries
- Editing transaction records
- Simulating unauthorized modifications

The forensic engine successfully detected integrity mismatches and marked affected files as tampered on the dashboard.

The dashboard generated warning alerts immediately after tampering detection.

Dashboard Testing

Dashboard testing verifies whether the Streamlit interface correctly displays forensic monitoring information and cybersecurity alerts.

The dashboard was tested for:

- Threat visualization
- File statistics display
- Alert generation
- Navigation functionality
- Report download operations

The testing confirmed that the dashboard provided accurate real-time monitoring information.

Report Generation Testing

The report generation module was tested to verify whether forensic reports were generated correctly in CSV format.

The reports successfully included:

- File names
- File paths
- SHA-256 hash values Integrity
- status
- Timestamps
- Suspicious activity indicators

The generated reports were successfully downloaded and preserved for forensic analysis.

Attack Simulation Testing

Attack simulation testing validates whether the system can detect simulated cyberattacks and unauthorized modifications.

The testing process involved:

- Injecting malicious entries Modifying integrity
- transaction files Triggering
- violations

The forensic monitoring engine successfully identified all simulated attacks and updated the dashboard with real-time alerts.

Advantages of Testing

The testing process provided several advantages:

- Improved system reliability Accurate
- integrity verification Better tampering
- detection
- Faster forensic analysis
- Improved cybersecurity monitoring
- Reliable dashboard visualization

Testing confirmed that the project successfully performs automated forensic monitoring, integrity verification, suspicious activity detection, dashboard visualization, and evidence preservation within the simulated retail environment.

Testing Results Summary				
Test Case	Expected Result	Actual Result	Status	
 Product Addition	Product added successfully	Success	Pass	
 Transaction Recording	Transaction stored correctly	Success	Pass	
 File Monitoring	File changes detected	Success	Pass	
 SHA-256 Verification	Integrity verified correctly	Success	Pass	
 Tampering Detection	Unauthorized changes identified	Success	Pass	
 Dashboard Alerts	Alerts displayed correctly	Success	Pass	
 CSV Report Generation	Report downloaded successfully	Success	Pass	
 Attack Simulation	Threat detected correctly	Success	Pass	

2.9 Advantages of the Proposed System

The project “Cloud Forensics Automation for Retail Breaches” provides several advantages in the areas of cybersecurity monitoring, digital forensic investigation, integrity verification, and suspicious activity detection. The system is designed to automate forensic analysis within a simulated retail environment while improving cybersecurity visibility and evidence preservation. Unlike traditional retail systems that mainly focus on billing and transaction management, the proposed system integrates automated forensic monitoring and tampering detection capabilities into the operational workflow. This improves system security and enables faster detection of cyber incidents.

The major advantages of the proposed system are described below.

Automated Forensic Monitoring

One of the major advantages of the system is automated forensic monitoring. The forensic engine continuously scans files stored inside monitored directories without requiring manual investigation.

The automated monitoring process helps:

- Detect file modifications quickly
- Reduce manual effort
- Improve investigation efficiency
- Monitor suspicious activities continuously

This improves overall cybersecurity visibility within the retail environment. Real- Time Tampering Detection

The system provides real-time tampering detection using SHA-256 integrity verification mechanisms. Whenever a monitored file is modified, the forensic engine immediately detects integrity mismatches and updates the dashboard with warning alerts.

This advantage helps:

- Identify unauthorized modifications instantly
- Reduce incident response time
- Improve threat detection accuracy
- Prevent unnoticed data manipulation

Real-time monitoring significantly improves cybersecurity response capabilities.

Improved Integrity Verification

The project uses SHA-256 cryptographic hashing to verify file integrity accurately. The integrity verification mechanism:

- Generates unique digital fingerprints
- Detects even small file modifications
- Prevents unnoticed tampering Improves
- evidence authenticity

This helps investigators identify compromised files during forensic investigations.

Centralized Dashboard Monitoring

The Streamlit dashboard provides centralized monitoring and real-time visualization of system security status.

The dashboard allows investigators to:

- Monitor file integrity
- Analyze suspicious activities
- View threat alerts
- Run forensic scans
- Download investigation reports

Centralized monitoring improves operational efficiency and simplifies cyber incident analysis.

Faster Digital Forensic Investigation

The automated forensic monitoring engine reduces the time required for manual evidence analysis.

The system helps investigators:

- Identify suspicious files quickly
- Analyze integrity violations efficiently Preserve
- digital evidence
- Generate investigation reports automatically

This improves forensic investigation speed and reduces operational complexity.

Improved Evidence Preservation

The system maintains activity logs and generates downloadable forensic reports in CSV format.

The evidence preservation process helps:

Attack Simulation Capability

The attack simulation module demonstrates how cyberattacks affect retail transaction environments.

This functionality helps:

- Validate forensic monitoring logic
- Demonstrate tampering detection
- Simulate insider attacks
- Improve cybersecurity understanding

The attack simulation feature makes the project suitable for cybersecurity learning and academic demonstrations.

Low Implementation Cost

The project is developed using open-source technologies such as:

- Python
- Flask
- Streamlit SHA-256
- 256 libraries

No expensive software or specialized infrastructure is required, making the project cost-effective and suitable for academic implementation.

User-Friendly Interface

The Streamlit dashboard provides a simple and easy-to-use interface for investigators and administrators.

The interface helps users:

- Navigate easily
- Monitor alerts efficiently
- Analyze reports quickly
- Perform scans without complexity

This improves usability and simplifies forensic monitoring operations.

Advantages Summary

Advantage	Benefit
1 Automated Monitoring	Continuous forensic analysis
2 Real-Time Detection	Faster threat identification
3 SHA-256 Verification	Accurate integrity checking
4 Dashboard Monitoring	Centralized security visibility
5 Evidence Preservation	Improved forensic investigation
6 Attack Simulation	Practical cybersecurity testing
7 Low Cost	Cost-effective implementation
8 User-Friendly Interface	Simplified monitoring operations

The proposed system successfully improves cybersecurity monitoring, digital forensic analysis, integrity verification, evidence preservation, and tampering detection within retail environments while providing a practical platform for forensic investigation and cybersecurity education.

3. Implementation

Implementation is the process of converting the designed system architecture into a working application capable of performing real-time forensic monitoring, integrity verification, tampering detection, dashboard visualization, and report generation. In the project “Cloud Forensics Automation for Retail Breaches,” the implementation phase focuses on developing a practical cybersecurity platform capable of monitoring retail transaction files and identifying unauthorized modifications automatically.

The project implementation combines multiple software components such as the retail transaction application, forensic monitoring engine, SHA-256 integrity verification system, Streamlit dashboard, attack simulation module, and forensic report generation system. Each component is implemented separately and integrated together to form a complete digital forensic monitoring platform.

The implementation process follows a modular development approach where individual functionalities are developed, tested, and integrated step by step. This modular implementation improves maintainability, debugging efficiency, scalability, and system reliability.

The implementation phase of the project is divided into the following major modules:

- Retail Application Implementation
- File Monitoring Implementation
- SHA-256 Integrity Verification Implementation
- Tampering Detection Implementation Dashboard Implementation
- Attack Simulation Implementation
- Activity Logging Implementation
- CSV Report Generation Implementation

Each implementation module contributes to the overall functionality of the system.

Retail Application Implementation

The retail application is implemented using the Flask framework and acts as the transaction generation environment of the project. The application simulates retail operations such as:

- Product management
- Customer purchases
- Sales recording

- Billing operations
- Inventory updates

Whenever a customer purchase is recorded, transaction details are stored inside monitored files such as CSV transaction records and activity logs.

The retail application generates:

- Customer name Product
- name
- Billing amount
- Transaction timestamps
- Sales records

These files later become evidence sources for forensic monitoring and integrity verification.

The Flask framework is selected because:

- It is lightweight
- Easy to implement
- Supports rapid web development
- Integrates easily with Python modules

The retail application successfully simulates real-world retail transaction environments.

File Monitoring Implementation

The file monitoring module continuously scans all files stored inside monitored directories.

The implementation uses Python file handling mechanisms to analyze:

- File existence
- File modification time
- File metadata
- File size
- File access timestamps

The monitoring engine automatically identifies newly added files and detects modifications performed on existing files.

The implementation process includes:

1. Selecting monitored directories
2. Scanning all files recursively
3. Extracting metadata information
4. Storing monitoring results
5. Forwarding files for integrity verification

The monitoring system acts as the evidence collection mechanism of the project.

SHA-256 Integrity Verification Implementation

The SHA-256 integrity verification system is implemented using Python's hashlib library.

This module generates cryptographic SHA-256 hash values for all monitored files.

The implementation process includes:

- Reading file content
- Generating SHA-256 hashes
- Storing hash values inside JSON files
- Comparing current and previous hashes
- Detecting integrity violations

The SHA-256 hashing algorithm produces unique digital fingerprints for every file. Even a small change in file content generates a completely different hash value.

The implementation successfully identifies unauthorized modifications accurately.

Tampering Detection Implementation

The tampering detection module is implemented by comparing current file hashes with previously stored hash values.

The detection process operates as follows:

1. Generate current hash value
2. Retrieve previous hash
3. Compare both values
4. Detect mismatches
5. Mark affected files as tampered

If integrity mismatches are detected:

- Threat alerts are generated
- Dashboard status changes to compromised
- Suspicious files are recorded
- Investigation logs are updated

This implementation enables real-time tampering detection within the monitored environment.

Dashboard Implementation

The forensic dashboard is implemented using the Streamlit framework. The dashboard acts as the Security Operations Center (SOC) interface of the project. The dashboard displays:

- Total monitored files
- Tampered files
- Suspicious activities
- Threat alerts
- Monitoring statistics
- Activity logs

The dashboard also provides operational controls for:

- Running forensic scans

- Simulating attacks
- Downloading reports
- Monitoring file integrity

Streamlit is selected because:

- It simplifies dashboard creation
- Supports real-time visualization
- Provides interactive interfaces
- Integrates easily with Python

The dashboard improves cybersecurity visibility and simplifies forensic analysis.

Attack Simulation Implementation

The attack simulation module demonstrates how attackers may manipulate retail transaction files and activity logs.

The implementation performs:

- Malicious entry injection
- Transaction modification
- File tampering simulation
- Integrity violation triggering

Example simulated attacks include:

- Unauthorized transaction editing
- Fake billing insertion
- Malicious activity log injection

The forensic engine successfully detects these modifications through integrity verification mechanisms.

Activity Logging Implementation

The activity logging module records important monitoring activities and forensic operations performed within the system.

The logs record:

- File scan operations
- Dashboard access
- Tampering incidents
- Attack simulations
- Monitoring alerts

The logging implementation improves:

- Investigation traceability
- Evidence preservation
- Security accountability

Activity logs help investigators analyze system behavior during cyber incidents.

CSV Report Generation Implementation

The forensic report generation module creates downloadable CSV reports containing investigation results.

The reports include:

- File names
- File paths
- SHA-256 hash values Integrity status
- Modification timestamps Suspicious activity indicators

The implementation uses Python CSV handling generation.

The generated reports help:

- Preserve digital,evidence Support, forensic analysis ,Improve incident documentation

Overall Implementation Workflow

The implementation workflow of the project operates as follows:

- Retail transactions are generated through the Flask application. Files are stored inside monitored directories.
- The monitoring engine scans all files continuously.
- SHA-256 hashes are generated for integrity verification. Current hashes are compared with stored values.
- Unauthorized modifications are detected automatically. Dashboard alerts are generated for investigators.
- CSV forensic reports are created for evidence analysis.

The implementation phase successfully converts the proposed forensic monitoring architecture into a practical cybersecurity platform capable of detecting retail data breaches, preserving digital evidence, and improving forensic investigation efficiency.

4. Graphical User Interface (GUI)

The Graphical User Interface (GUI) of the project “Cloud Forensics Automation for Retail Breaches” is designed to provide a simple, interactive, and user-friendly environment for performing forensic monitoring, integrity verification, tampering detection, and cybersecurity analysis. The GUI helps investigators and administrators monitor retail transaction activities and identify suspicious modifications efficiently without requiring complex command-line operations.

The project contains two major interfaces:

- Retail Management Interface Cloud
- Retail Forensics Dashboard

The Retail Management Interface simulates retail operations such as adding products and recording customer transactions. The Cloud Retail Forensics Dashboard acts as the Security Operations Center (SOC) interface that provides centralized forensic monitoring and real-time cybersecurity visualization.

The GUI is developed using:

- Flask web framework
- HTML interface components
- Streamlit dashboard framework Interactive
- buttons and visualization tools

The interface design focuses on simplicity, visibility, accessibility, and operational efficiency.

4.1 Retail Management Interface

The Retail Management Interface acts as the simulated retail transaction environment of the project. This interface allows users to generate transaction records and operational files that are later analyzed by the forensic monitoring engine.

The retail interface provides functionalities such as:

- Adding new products
- Recording customer purchases
- Generating transaction logs
- Maintaining transaction history

The interface displays transaction information including:

- Timestamp
- Customer name
- Product name
- Billing amount

The generated transaction records are automatically stored inside monitored directories for forensic analysis.

The interface contains:

- “Add New Product” button
- “Record Sale” button
- Transaction history table
- Billing information display

This interface simulates how real retail systems continuously generate operational data.

4.2 Cloud Retail Forensics Dashboard

The Cloud Retail Forensics Dashboard acts as the central forensic monitoring interface of the project. The dashboard is developed using Streamlit and provides real-time cybersecurity visibility.

The dashboard displays:

- System security status
- Total monitored files Tampered
- files
- Suspicious files
- Threat alerts
- File status distribution graphs
- Activity logs

The dashboard automatically updates whenever:

- Files are modified Integrity
- violations occur Tampering
- is detected New scans are
- performed

The dashboard helps investigators quickly identify suspicious activities and compromised files.

4.3 Dashboard Functional Buttons

The forensic dashboard contains several operational buttons that allow investigators to perform different forensic operations.

Run Forensics Scan

The “Run Forensics Scan” button initiates the forensic monitoring engine.

This operation:

- Scans monitored files
- Generates SHA-256 hashes
- Performs integrity verification
- Detects suspicious modifications

The scan results are displayed immediately on the dashboard.

Simulate Attack (Tamper)

The “Simulate Attack” button demonstrates cyberattack scenarios by modifying transaction files intentionally.

The simulation:

- Injects malicious entries
- Changes transaction records
- Triggers integrity violations
- Generates threat alerts

This feature demonstrates how forensic systems detect unauthorized modifications.

Download Forensic Report

The “Download Forensic Report” button generates downloadable CSV investigation reports.

The reports contain:

- File information
- Hash values
- Integrity status
- Modification timestamps
- Suspicious activity details

This feature helps preserve forensic evidence.

4.4 GUI Design Features

The GUI design focuses on simplicity and accessibility.

The major design features include:

- Dark-themed forensic dashboard Real-time threat visualization Centralized monitoring interface Interactive operational buttons Simple navigation controls Graph-based statistics visualization

The design improves usability and enhances cybersecurity visibility.

4.5 GUI Workflow

The GUI workflow operates through the following process: User records transactions through the retail interface. Transaction files are stored inside monitored directories. The forensic dashboard scans the monitored files. SHA-256 hashes are generated and verified. Tampered files are identified automatically.

Threat alerts are displayed on the dashboard. Investigation reports are generated and downloaded.

The GUI simplifies forensic monitoring and allows investigators to analyze cybersecurity incidents efficiently.

4.6 Advantages of GUI

The GUI provides several advantages:

- Simplified forensic monitoring
- Easy dashboard navigation
- Real-time threat visibility
- Faster investigation workflow Improved user interaction Centralized cybersecurity monitoring

The Graphical User Interface successfully improves accessibility, visualization, and operational efficiency within the forensic monitoring platform while providing practical cybersecurity analysis capabilities for retail breach investigations.

5. Results and Discussion

The Results and Discussion section explains the operational outcomes, forensic monitoring performance, integrity verification results, tampering detection accuracy, and overall effectiveness of the project “Cloud Forensics Automation for Retail Breaches.” The results obtained during implementation and testing confirm that the system successfully performs automated forensic monitoring, identifies unauthorized modifications, preserves digital evidence, and provides real-time cybersecurity visibility within the simulated retail environment.

The project integrates retail transaction simulation with automated forensic analysis mechanisms capable of continuously monitoring transaction files and identifying integrity violations through SHA-256 cryptographic hashing. The generated results demonstrate that the system can effectively detect suspicious activities and unauthorized modifications within monitored directories.

The forensic dashboard provides centralized monitoring and displays real-time alerts whenever integrity mismatches occur. The generated reports and activity logs also help preserve evidence for digital forensic investigation.

Successful Retail Transaction Generation

The retail management application successfully simulated real-world retail operations by generating:

- Product records
- Customer transaction details
- Billing information
- Inventory updates
- Activity logs

The generated transaction files were automatically stored inside monitored directories and later analyzed by the forensic monitoring engine.

The retail application successfully created operational data suitable for digital forensic analysis and cybersecurity monitoring.

Successful File Monitoring

The forensic monitoring engine successfully scanned files stored inside monitored directories continuously.

The monitoring system successfully performed:

- File detection
- Metadata extraction
- Timestamp analysis
- File modification tracking
- Evidence collection

The monitoring engine accurately identified newly added files and detected modifications performed on existing transaction records.

This confirms that the monitoring mechanism operated successfully within the forensic environment.

Successful SHA-256 Integrity Verification

The SHA-256 integrity verification mechanism successfully generated unique cryptographic hash values for all monitored files.

The integrity verification process confirmed that:

- Every file generated a unique hash value
- Small file modifications changed the hash completely
- Unauthorized modifications were accurately identified
- Integrity verification operated reliably

The SHA-256 hashing mechanism successfully preserved file authenticity and improved forensic analysis accuracy.

Successful Tampering Detection

The project successfully detected unauthorized modifications within transaction files and activity logs.

Tampering detection was validated through:

- Simulated file modification
- Malicious entry injection
- Transaction record editing
- Activity log manipulation

Whenever a file was modified:

- The hash value changed
- Integrity verification failed
- The file was marked as tampered
- Threat alerts were displayed on the dashboard

The system successfully identified integrity violations in real time.

Dashboard Performance Results

The Streamlit forensicdashboardsuccessfully provided centralized cybersecurity monitoring and visualization.

The dashboard accurately displayed:

- Total monitored files
- Tampered files Threat
- alerts
- File status statistics
- Activity logs
- Investigation results

The dashboard interface improved visibility and simplified forensic investigation operations.

The real-time alert system successfully updated the dashboard whenever suspicious activities were detected.

Attack Simulation Results

The attack simulation module successfully demonstrated how cyberattacks affect retail transaction environments.

The simulated attacks included:

- Fake transaction insertion
- Transaction value modification
- Malicious activitylog injection
- Unauthorized file editing

The forensic monitoring engine successfully identified all simulated attacks and updated the dashboard with warning alerts.

This confirmed the effectiveness of the tampering detection mechanism.

CSV Report Generation Results

The CSVreportgenerationmodulesuccessfully created downloadable forensic investigation reports.

The generated reports contained:

- File names
- File paths
- SHA-256 hash values
- Integrity verification results
- Modification timestamps
- Suspicious activity indicators

The reports successfully preserved forensic evidence for future analysis.

Discussion

The overall project successfully demonstrates how automated forensic monitoring can improve cybersecurity visibility and integrity verification within retail environments.

The project provides:

- Continuous forensic monitoring Real-
- time tampering detection Centralized
- dashboard visualization Automated
- evidence preservation Improved
- investigation efficiency

The use of SHA-256 hashing significantly improved integrity verification accuracy and allowed the system to identify even small modifications within monitored files.

The integration of Streamlit-based dashboard visualization simplified cybersecurity monitoring and improved accessibility for investigators and administrators.

The attack simulation functionality also demonstrated practical cyberattack scenarios and validated the effectiveness of the forensic monitoring engine.

Although the project operates within a simulated retail environment, the implemented methodology can be expanded for real-world cybersecurity monitoring and cloud forensic analysis systems.

Result Summary

Operation	Result
 Retail Transaction Simulation	Successful
 File Monitoring	Successful
 SHA-256 Integrity Verification	Successful
 Tampering Detection	Successful
 Dashboard Visualization	Successful
 Attack Simulation	Successful
 CSV Report Generation	Successful

6. Conclusion

The project “Cloud Forensics Automation for Retail Breaches” was successfully developed and implemented as an automated digital forensic monitoring platform for simulated retail environments. The system successfully combines retail transaction simulation, forensic monitoring, SHA-256 integrity verification, tampering detection, dashboard visualization, attack simulation, and forensic report generation into a single unified cybersecurity platform. The project demonstrates how modern retail systems can improve cybersecurity visibility and protect sensitive transaction records through automated forensic analysis techniques. Traditional retail systems often lack continuous integrity verification and real-time unauthorized modifications, and digital evidence manipulation. The proposed system successfully addresses these challenges by continuously monitoring files and detecting suspicious activities automatically.

The implementation of SHA-256 cryptographic hashing significantly improved the accuracy of file integrity verification. Even small modifications within transaction files generated completely different hash values, allowing the system to identify unauthorized changes effectively. The forensic monitoring engine successfully detected simulated cyberattacks and updated the dashboard with real-time alerts whenever integrity violations occurred.

The Streamlit-based forensic dashboard provided centralized cybersecurity monitoring and simplified forensic investigation procedures. Investigators could monitor tampered files, suspicious activities, integrity statistics, and investigation logs through an interactive interface. The dashboard improved operational visibility and made forensic analysis easier and more efficient.

The attack simulation module successfully demonstrated how attackers may manipulate retail transaction records and operational logs. The system accurately identified malicious modifications and preserved evidence through downloadable forensic reports and activity logs. This validated the effectiveness of the forensic monitoring engine and demonstrated practical cybersecurity investigation techniques.

The project also successfully achieved the following objectives:

- Automated forensic monitoring
- Real-time tampering detection
- SHA-256 integrity verification
- Centralized dashboard visualization
- Attack simulation
- Activity logging
- CSV forensic report generation
- Digital evidence preservation

The project proves that automated forensic monitoring systems can improve cybersecurity analysis and reduce the difficulty detecting unauthorized modifications within retail environments.

The developed platform serves as an effective academic model for understanding digital forensics, cybersecurity monitoring, integrity verification, and incident investigation techniques.

Overall, the project successfully fulfills its goal of implementing a practical and efficient forensic monitoring platform capable of detecting retail breaches, preserving

digital evidence, and improving cybersecurity visibility within transaction environments.

7. Future Scope

The project “Cloud Forensics Automation for Retail Breaches” provides a strong foundation for future improvements and advanced cybersecurity research. Although the current implementation successfully performs automated forensic monitoring and tampering detection within a simulated retail environment, several advanced features can be added in the future to improve scalability, intelligence, automation, and real- world deployment capabilities.

The future scope of the project includes the integration of advanced cybersecurity technologies, cloud-based monitoring systems, artificial intelligence techniques, and real-time threat intelligence platforms.

Several future enhancements are described below.

Cloud Integration

The current project mainly operates within a local simulated environment. In the future, the system can be integrated with cloud platforms such as:

- AWS
- Microsoft Azure
- Google Cloud Platform

Cloud integration would allow:

- Real-time cloud forensic monitoring
- Distributed evidence collection
- Secure cloud storage analysis
- Enterprise-level deployment

This would improve scalability and real-world applicability.

AI-Based Threat Detection

Artificial Intelligence and Machine Learning algorithms can be integrated into the system to improve suspicious activity analysis and anomaly detection.

AI integration can help:

- Detect abnormal transaction patterns
- Predict cyberattacks
- Identify insider threats
- Improve automated investigation

Machine learning models can enhance forensic monitoring accuracy significantly.

Real-Time Alert Notification System

Future versions of the project can include automated alert notification systems. The system can send:

- Email alerts
- SMS notifications
- Mobile alerts
- Administrator warnings

This would improve incident response speed and reduce attack detection delays.

DatabaseIntegration

The current project mainly uses CSV and file-based storage mechanisms. Future implementations can integrate:

- MySQL
- PostgreSQL
- MongoDB
- Cloud databases

Database integration would improve :

- Large-scale evidence storage Query
- performance
- Transaction management
- Scalability

AdvancedDashboardAnalytics

The dashboard can be improved with advanced analytics and visualization features such as:

- Real-time graphs
- Threat heatmaps
- Incident timelines
- Statistical analysis
- Predictive monitoring

Advanced analytics would improve cybersecurity visibility and investigation efficiency.

Multi-User Access Control

Future versions can support:

- Role-based access control
- Multi-user authentication
- Investigator management
- Administrative permissions

This would improve security and enterprise usability.

Live Network Monitoring

The project can be extended to monitor:

- Network traffic
- Suspicious IP addresses
- Unauthorized network access
- Cloud communication logs

This would transform the system into a more advanced Security Operations Center (SOC) platform.

Automated IncidentResponse

Future implementations may include automated response mechanisms capable of:

- Isolating compromised files
- Blocking suspicious activities
- Triggering defensive actions
- Generating automated investigation workflows

This would improve cybersecurity automation capabilities.

Mobile Dashboard Support

A mobile-friendly forensic dashboard can be developed for:

- Remote monitoring
- Mobile threat analysis
- Alert management
- Portable investigation access

This would improve accessibility and operational flexibility.

Future Scope Summary

Enhancement	Benefit	Enhancement	Benefit
 Cloud Integration	Enterprise scalability	 Multi-User Access	Improved security management
 AI Threat Detection	Smarter anomaly analysis	 Network Monitoring	Expanded cybersecurity coverage
 Real-Time Alerts	Faster incident response	 Automated Response	Faster mitigation
 Database Integration	Improved data management	 Mobile Support	Remote accessibility
 Advanced Analytics	Better threat visualization		

8. References

- 1.[Python Official Documentation](#)
- 2.[Flask Official Documentation](#)
- 3.[Streamlit Documentation](#)
- 4.[OWASP Foundation](#)
- 5.[National Institute of Standards and Technology \(NIST\)](#)
- 6.[Python hashlib Documentation](#)
- 7.[Digital Forensics Research Workshop \(DFRWS\)](#)
- 8.[Cybersecurity and Infrastructure Security Agency \(CISA\)](#)
- 9.[SQLite Documentation](#)
- 10.[GitHub Documentation](#)
- 11.[SHA-256 Cryptographic Hash Function Information](#)
- 12.[Pandas Documentation](#)
- 13.[Cyber Forensics Basics - IBM Security](#)
- 14.[Cloud Security Alliance](#)
- 15.[Streamlit Community Cloud Documentation](#)

9. Appendix

Appendix A – Project Folder Structure

```
Cloud-Forensics-Automation-for-Retail-Breaches/ |
|— monitored_data/
|— mock_evidence/
|— cloud_storage/
|— app.py
|— retail_app.py
|— retail_evidence_collector.py
|— report.csv
|— has h_store.json
|— access_log.txt
```


└─ requirements.txt
└─ README.md

Appendix B – Software Requirements

Software	Purpose
Python	Backend development
Flask	Retail web application
Streamlit	Dashboard interface
VS Code	Development environment
GitHub	Version control
Windows OS	Execution platform

Appendix C – Hardware Requirements

Hardware Component	Specification
Processor	Intel i3 or higher
RAM	Minimum 4 GB
Storage	Minimum 20 GB
Operating System	Windows 10/11
Internet	Required for GitHub and dependencies

Appendix D – GitHub Repository

Project Repository:

<https://github.com/lavatoz/Cloud-Forensics-Automation-for-Retail-Breaches>

Appendix E – Common Commands Used

- Run Retail Application

```
python retail_app.py
```

- Run Streamlit Dashboard

```
streamlit run app.py
```

- Install Dependencies

```
pip install -r requirements.txt
```

- GitHub Push Commands

```
git add .
```

```
git commit -m "Project Update"
```

```
git push origin main
```